WEEKLY TASK 1 REPORT

NumPy Matrix Operations

Full Name	Asma Noonari
Internship Domain	AI/ML
Email Address	asmanoonari18@gmail.com
Phone Number	03103277171
Task	Weekly Task 1

Project

Basic Matrix Creation and Mathematical Operations Using NumPy

1. Introduction

This Weekly Task 1 demonstrates the use of Python and the NumPy library to create, display, and perform mathematical operations on two-dimensional arrays (matrices). The task focuses on fundamental matrix operations including addition, subtraction, element-by-element multiplication, matrix multiplication, scalar multiplication, summation, and average calculation.

2. Objective

- Create two matrices using NumPy arrays.
- Display the matrices clearly in the console.
- Perform addition and subtraction between matrices.
- Perform element-by-element multiplication.
- Perform matrix multiplication using the transpose of the second matrix.
- Multiply a matrix by a scalar value.
- Calculate the sum and average of all elements in a matrix.
- Verify the program results using the generated console output.

3. Tools and Technologies Used

Item	**Details**
Programming Language	Python
Library	NumPy
Development Environment	Python-compatible IDE/terminal
Input	Two 2 × 3 matrices
Output	Console results for each operation

4. Matrix Data

Matrix A is defined as:

[[1, 2, 3],
 [4, 5, 6]]

Matrix B is defined as:

[[7, 8, 9],
 [10, 11, 12]]

Both matrices have dimensions 2 × 3. This makes them directly compatible for element-wise addition, subtraction, and multiplication.

5. Code Implementation

```
import numpy as np

# Create two grids (matrices)
A = np.array([
    [1, 2, 3],
    [4, 5, 6]
])
```

```
B = np.array([
    [7, 8, 9],
    [10, 11, 12]
])

# Display the matrices
print("Matrix A:")
print(A)

print("\nMatrix B:")
print(B)

# Basic math
print("\nA + B:")
print(A + B)

print("\nA - B:")
print(A - B)

# Element-by-element multiplication
print("\nA * B (element-by-element):")
print(A * B)

# Matrix multiplication requires compatible dimensions.
# Here, transpose B from 2x3 to 3x2.
print("\nA @ B.T (matrix multiplication):")
print(A @ B.T)

# Other useful operations
print("\nA multiplied by 2:")
print(A * 2)

print("\nSum of all values in A:")
print(np.sum(A))

print("\nAverage of values in A:")
print(np.mean(A))
```

6. Explanation of the Implementation

Importing NumPy: import numpy as np loads NumPy and gives it the short name np, making its array and mathematical functions convenient to use.

Creating matrices: np.array() converts the nested Python lists into NumPy arrays representing Matrix A and Matrix B.

Addition: A + B adds corresponding elements from the two matrices.

Subtraction: A - B subtracts each element of B from the corresponding element of A.

Element-wise multiplication: A * B multiplies corresponding elements rather than performing conventional matrix multiplication.

Matrix multiplication: A @ B.T uses the @ operator for matrix multiplication. B.T changes B from 2 × 3 to 3 × 2, so the multiplication A (2 × 3) × B.T (3 × 2) is valid.

Scalar multiplication: A * 2 multiplies every element of Matrix A by 2.

Sum: np.sum(A) adds all six values in Matrix A.

Average: np.mean(A) calculates the arithmetic mean of all six values in Matrix A.

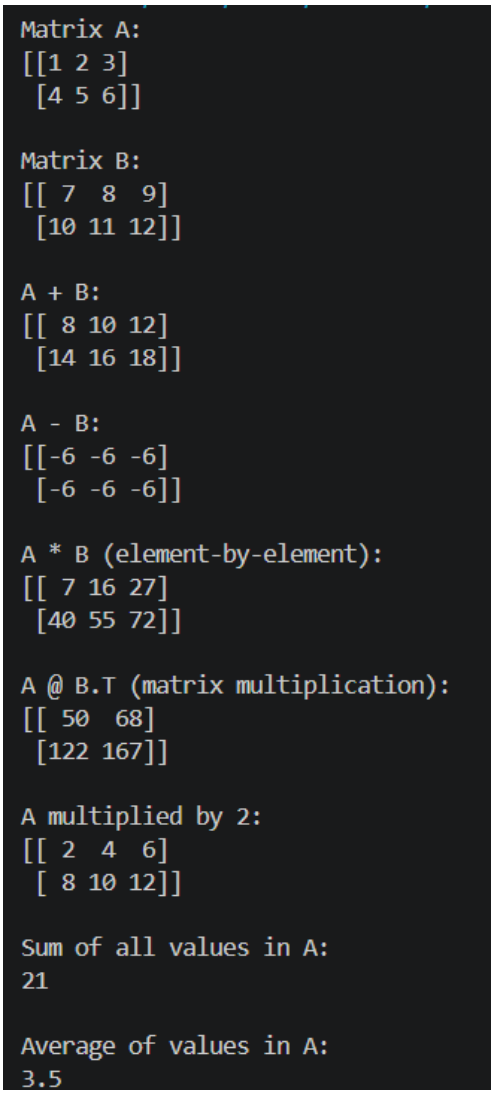
7. Results and Output Analysis

The program executed successfully and produced the expected results. The output shows that all operations were performed correctly.

Operation	Result
A + B	[[8, 10, 12], [14, 16, 18]]
A - B	[[-6, -6, -6], [-6, -6, -6]]
A * B	[[7, 16, 27], [40, 55, 72]]
A @ B.T	[[50, 68], [122, 167]]
A * 2	[[2, 4, 6], [8, 10, 12]]
Sum of A	21
Average of A	3.5

8. Screenshot of Program Output

The following screenshot provides visual evidence of the completed execution and the resulting matrix calculations.



9. Learning Outcomes

- Understood how NumPy arrays can represent matrices.
- Learned the difference between element-wise multiplication (*) and matrix multiplication (@).
- Practiced matrix dimension compatibility and the use of transpose (B.T).
- Used NumPy aggregation functions such as np.sum() and np.mean().
- Verified numerical results through console output.

10. Conclusion

Weekly Task 1 was completed by implementing a Python program with NumPy for basic matrix operations. The program successfully created two matrices, performed the required calculations, and displayed the results. The task provides a practical foundation for working with arrays and matrices, which are important concepts in data analysis, machine learning, and scientific computing.

11. Submission Note

Before submission, replace the front-page placeholders with the required personal details. The assignment instructions provided the file-naming requirement but did not include the actual naming template after the colon, so the working filenames used for this report are Weekly_Task_1_Report.docx and Weekly_Task_1_Report.pdf.